IFC-CAMBORIÚ
CURSO SUPERIOR DE TECNOLOGIA EM SISTEMAS PARA INTERNET
ALGORITMOS E TÉCNICAS DE PROGRAMAÇÃO
Lista de Exercícios - Modularização
PROF. RAFAEL DE MOURA SPERONI
Acadêmico: Bernardo Werner da Rocha

1. Criar uma função que calcule e retorne o MAIOR entre dois valores recebidos como parâmetro. Um algoritmo para testar tal função deve ser criado.

```python
def maiorNum():
  if n1 > n2:
    return n1
  else:
    return n2

def teste():
  global n1
  global n2
  n1 = float(input("Insira o 1° n°: "))
  n2 = float(input("Insira o 2° n°: "))
  print(f"O maior número é: {maiorNum()}")

teste()
```

2. Escreva um procedimento que receba um número inteiro e imprima o mês correspondente ao número. Por exemplo, 2 corresponde à “fevereiro”. O procedimento deve mostrar uma mensagem de erro caso o número recebido não faça sentido.
   Gere também um algoritmo que leia um valor e chame o procedimento criado.

```python
def lervalor():
  global mesNumero
  mesNumero = int(input("Insira o mês: "))
  mesExtenso()

def mesExtenso():
  meses = ["Janeiro", "Fevereiro", "Março", "Abril", "Maio", "Junho", "Julho", "Agosto", "Setembro", "Outubro", "Novembro", "Dezembro"]
  if mesNumero ≥1 and mesNumero ≤ 12:
    print(meses[mesNumero - 1])
  else:
    print("Erro: Insira um número inteiro entre 1 e 12")

lervalor()
```

3. Escreva um procedimento que gere um cabeçalho para um relatório. Esse procedimento deve receber um literal (string, ou cadeia de caracteres) como parâmetro. O cabeçalho tem a seguinte forma:
   ==========
   IFC – Camboriú
   Curso Superior de Tecnologia em Sistemas para Internet
   Disciplina de Algoritmos e Técnicas de Programação
   Nome: **<seu_nome>**
   ========== ,
   onde **<seu_nome>**, corresponde ao parâmetro passado.

```python
def cabecalhoPadrao():
  nome = str(input("Informe seu nome: "))
  cabecalho = str(f"{==========\nIFC - Camboriú\nCurso Superior de Tecnologia em Sistemas para Internet\nDisciplina de Algoritmos e Técnicas de Programação\nNome: {nome}\n==========}")
  print(cabecalho)

cabecalhoPadrao()
```

4. Escreva um procedimento que receba um número inteiro e o imprima na forma extensa. Por exemplo, para 1 a saída desejada é “Um”. A função deve ser capaz de gerar o extenso dos números de 0 até 10, inclusive. Caso um número não compatível seja recebido o procedimento deve mostrar uma mensagem de erro.
   Crie também um algoritmo que leia um valor inteiro e chame o procedimento criado acima para a impressão do número extenso.

```python
def digiteNum():
  global num
  num = int(input("Digite um número inteiro de 0 a 10: "))
  return num

def numExtenso():
  num
  numeros=["zero","um","dois","três","quatro","cinco","seis","sete","oito","nove","dez"]
  if num ⩾0 and num ⩽ 10:
    print(numeros[num])
  else:
    print("Número inválido")

digiteNum()
numExtenso()
```

5. Escreva um procedimento que receba um número natural e imprima os três primeiros caracteres do dia da semana correspondente ao número. Por exemplo, 7 corresponde à “SAB”. O procedimento deve mostrar uma mensagem de erro caso o número recebido não corresponda a um dia da semana.
   Gere também um algoritmo que utilize esse procedimento, chamando-o, mas antes lendo um valor para passagem de parâmetro.

```python
def diasSemana():
  diaSemana = ["SEG","TER","QUA","QUI","SEX","SAB","DOM"]
  if diaNum ⩾1 and diaNum ⩽ 7:
    print(diaSemana[diaNum-1])
  else:
    print("Dia invalido")

def diasNumericos():
  global diaNum
  diaNum = int(input(f"(Informe o dia da semana de 1 a 7)"))
  diasSemana()
  return diaNum
```

6. Escreva uma função que receba um número inteiro. Esta função deve verificar se tal número é primo. No caso positivo, a função deve retornar 1, caso contrário zero. Escreva também um algoritmo para testar tal função.

```python
def numPrimo():
    global num
    num = int(input("Digite um número: "))
    if num > 1:
       for i in range(2, num):
          if (num % i) == 0:
             return 0
       else:
          return 1

def teste():
  if numPrimo() == 1:
      print(num, "é um número primo")
  else:
      print(num, "não é um número primo")

teste()
```

7. Escreva uma função que receba dois números inteiros x e y. Essa função deve verificar se x é divisível por y. No caso positivo, a função deve retornar 1, caso contrário zero.
   Escreva também um algoritmo para testar tal função.

```python
def divisor():
  global x , y
  x = int(input("Digite o valor de x: "))
  y = int(input("Digite o valor de y: "))
  if x % y == 0:
    return 1
  else:
    return 0

def teste():
  if divisor() == 1:
    print("É divisível")
  else:
    print("Não é divisível")

teste()
```

8. Criar uma função que verifique quantas vezes um número inteiro x é divisível por um número inteiro y. A função deve retornar -1 caso não seja possível calcular. Escreva também um algoritmo para testar tal função.

```python
def quociente():
  x = int(input(f"(Digite o valor de x)"))
  y = int(input(f"(Digite o valor de y)"))
  if x % y ≠ 0:
    return -1
  else:
    return (f"O resultado da divisão é {x/y}")

def teste():
  print (quociente())

teste()
```

9. Criar uma função que receba um número inteiro e retorne o seu valor por extenso. Por exemplo, para 5 o valor de retorno desejado é “cinco”. A função deve ser capaz de gerar o valor por extenso para os 50 primeiros inteiros. Uma mensagem de erro deve ser mostrada caso um número fora dessa faixa seja recebido. Escreva também um algoritmo para testar a função.

```python
def numExtenso():
  num = int(input(f"Digite um número de 0 a 50: "))
  if 0 ≤ num ≤ 50:
      extenso = ["zero", "um", "dois", "três", "quatro", "cinco", "seis", "sete", "oito", "nove", "dez",
                 "onze", "doze", "treze", "quatorze", "quinze", "dezesseis", "dezessete", "dezoito", "dezenove",
                 "vinte", "vinte e um", "vinte e dois", "vinte e três", "vinte e quatro", "vinte e cinco",
                 "vinte e seis", "vinte e sete", "vinte e oito", "vinte e nove", "trinta", "trinta e um",
                 "trinta e dois", "trinta e três", "trinta e quatro", "trinta e cinco", "trinta e seis",
                 "trinta e sete", "trinta e oito", "trinta e nove", "quarenta", "quarenta e um", "quarenta e dois",
                 "quarenta e três", "quarenta e quatro", "quarenta e cinco", "quarenta e seis", "quarenta e sete",
                 "quarenta e oito", "quarenta e nove", "cinquenta"]
      return extenso[num]
  else:
      return "Número inválido."

def teste():
  print(numExtenso())

teste()
```

10. Escreva um procedimento que receba um número arábico inteiro e imprima o correspondente número em romano. Por exemplo, para 5 a saída desejada é “V”. A função deve ser capaz de gerar o número romano para os 50 primeiros inteiros. Uma mensagem de erro deve ser mostrada caso um número fora dessa faixa seja recebido.
    Crie também um algoritmo que leia um valor inteiro e chame o procedimento criado acima para a impressão do número romano.

```python
def numRomanos():
  romanos = ['I', 'II', 'III', 'IV', 'V', 'VI', 'VII', 'VIII', 'IX', 'X', 'XI',
            'XII', 'XIII', 'XIV', 'XV', 'XVI', 'XVII', 'XVIII', 'XIX', 'XX',
            'XXI', 'XXII', 'XXIII', 'XXIV', 'XXV', 'XXVI', 'XXVII', 'XXVIII',
            'XXIX', 'XXX', 'XXXI', 'XXXII', 'XXXIII', 'XXXIV', 'XXXV', 'XXXVI',
            'XXXVII', 'XXXVIII', 'XXXIX', 'XL','XLI','XLII','XLIII',"XLIV", "XLV",
            "XLVI","XLVII","XLVIII","XLIX","L",]
  if 1 ⩽ num ⩽ 50:
   return romanos[num-1]
  else:
      return "Número inválido."

def leitorNum():
  global num
  num = int(input(f"Digite um número de 1 a 50: "))
  print(numRomanos())

leitorNum()
```